

GAME DESIGN DOCUMENT (GDD)

Game Name: VR Shooting Arena	1
Technical Specs	1
Gameplay	2
Design Guidelines	2
Game Elements	2
User Interface	3
Game Flow	3
Win and Lose Conditions	3
Key Features	3
Audio	3
Game Screenshots	4
Repository	4
Used tools and websites	5
Code important parts explained	5

Game Name: VR Shooting Arena

Genre: VR Shooting Simulator / Arcade

Game Elements: Players shoot appearing targets trying to score as many hits as possible with a limited time.

Player: Single-player only

Technical Specs

Engine: Unreal Engine 5.7.1

Technical Form: 3D graphics, Deferred PBR rendering

Platform: PC (Windows)

Language: Blueprints

Input Devices: VR motion controllers, VR headset

Gameplay

Core Gameplay Loop

- Player shoots the Start button
- The round begins
- Targets appear randomly in the arena
- Player shoots targets to score points
- Time counts down
- Round ends when time reaches zero
- Final score is displayed
- Player can always try again by shooting the Start button again

Game rules

- Targets appear and disappear during the round
- Each hit target gives +1 to the score
- Targets disappear after being hit or after a set time
- The round duration is limited - 30 seconds

Controls

VR Controls

- **Aim** - VR controller orientation
- **Grab** - right controller grab button
- **Shoot** - trigger button
- **Start round** - Shoot the Start object
- **Reset position, restart** - menu button on left controller

Design Guidelines

Targets must appear, the score must be PROPERLY counted, and the time limit should exist. The environment is designed to support the experience while remaining visually neutral, ensuring the player's attention stays on the targets.

Game Elements

Player

- First-person VR perspective
- Stationary shooting area

Targets

- Appear randomly in predefined positions
- Can be hit by bullets (projectiles)
- Play sound effect on hit
- Disappear after hit or a previously set time

Weapon

- Can be picked up
- Shoots projectiles

Projectiles

- Spawned from weapon
- Detect collision with targets
- Trigger hit logic and sound

User Interface

UI Design

Only Text elements in the VR World to avoid motion-sickness from the static HUD.

UI Elements

- Remaining time
- Number of hits (score)
- Text 'Shoot to start'

Game Flow

Launch Game -> Load Arena -> Idle State -> Shoot Start Button -> Active Round -> Time Countdown -> End Round -> Final Score

Win and Lose Conditions

NONE, the game is projected as an endless score challenge, not a story-based experience.

Key Features

- VR based shooting mechanics
- Randomized target spawning
- Time-limited rounds
- Score tracking
- World-space UI for comfort
- Blueprint-only implementation

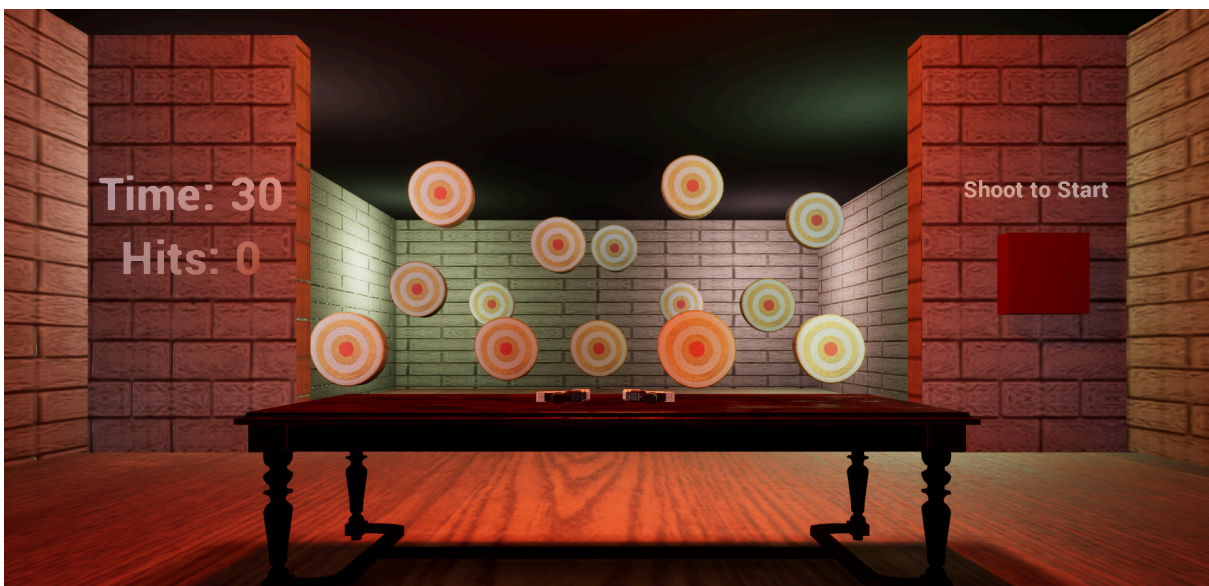
Audio

- Hit sound played when target is destroyed
- start sound when Player shoots Start Button

Inspirations and Ideas

- Aim training maps from CS:GO
- Valorant Practice Range

Game Screenshots



Repository

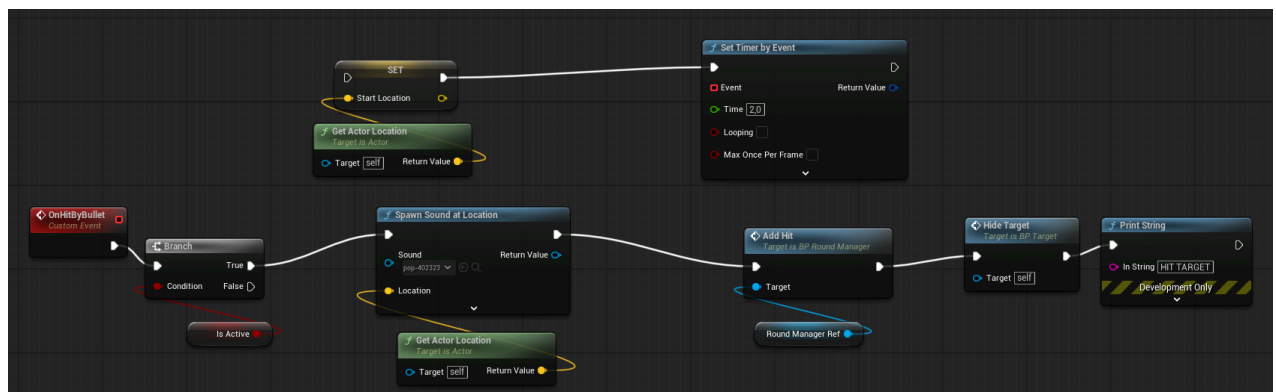
<https://github.com/Lesiaaa/VrShootingArena>

Used tools and websites

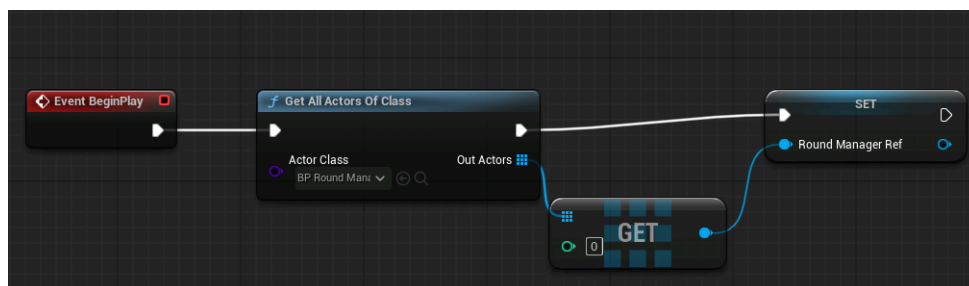
- <https://www.fab.com/listings/4266255a-9d72-4e05-aa40-8e3b46665b2f>
- <https://www.fab.com/listings/e43ac011-52db-492d-9619-63e642718618>
- <https://www.fab.com/listings/a72791f2-a9c8-49bd-a057-6cf4b0303b81>
- <https://www.fab.com/listings/f0cb0923-dc9d-43de-b3b7-7481bd26188e>
- <https://pixabay.com/sound-effects/search/start/>
- <https://pixabay.com/sound-effects/search/pop/>
- https://dev.epicgames.com/documentation/en-us/unreal-engine/building-virtual-worlds?application_version=4.27
- <https://www.youtube.com/@UnrealEngine/courses>
- https://www.youtube.com/watch?v=i3xNb5R_xos&t=923s
- chat gpt, for helping with learning Unreal Engine, refining code and tricky bugs :))

Code important parts explained

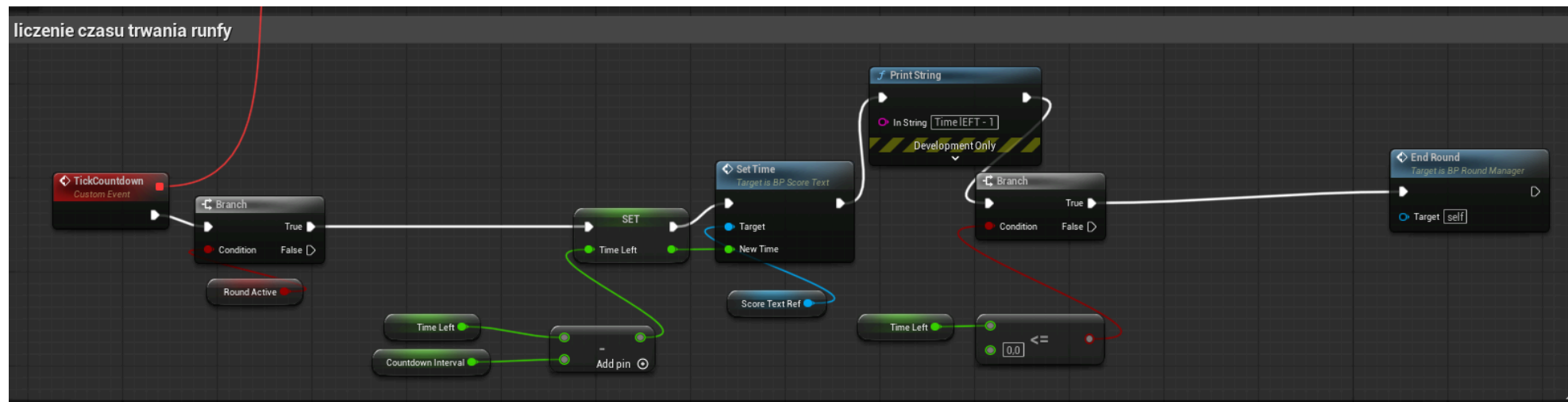
Target disappears after hitting it with a bullet. First, we check if the round is active, we spawn the sound effect, add one point to the score and we hide the target.



On start, each target searches and stores a reference to the Round Manager. It allows to report hits within the system.



TickCountdown Event is responsible for handling the round timer. It checks if the round is active, then Sets a new time using the Time Left - Countdown Interval (in this case 1 second). Then it checks whether the time is $\leq$ than 0, end if true, calls the end of the round.



Each new round is started by resetting the timers, so they do not double. Then we set all parameters back to previous settings - score to 0, Round time to 30 seconds etc.

